

Whip Dynamics: FEA Simulation and GNN Surrogate Modeling

LAU MeltFlow Project

March 14, 2026

Abstract

This report documents the development of a finite-element-style simulation of a bead chain (whip precursor) and the eventual training of a graph neural network (GNN) surrogate solver. We record the formulations attempted, what worked, what failed, and the rationale behind each design decision.

Contents

1	Introduction and Motivation	3
2	Problem Setup	3
2.1	Geometry and Initial Conditions	3
2.2	Graph Structure	3
3	Formulation Attempts	3
3.1	Attempt 1: Position Verlet with SHAKE Constraints	4
3.1.1	Formulation	4
3.1.2	Results	4
3.1.3	Why It Failed	4
3.1.4	Lessons Learned	4
3.2	Attempt 2: Symplectic Euler with Penalty Springs	5
3.2.1	Formulation	5
3.2.2	Walkthrough: Updating a Single Bead	5
3.2.3	Parameters	6
3.2.4	Stability Criterion	6
3.2.5	Results	6
3.2.6	Why It Works	7
4	Implementation	7
4.1	Code Structure	7
4.2	Data Export Format	7
5	Qt/C++ Implementation: Signal/Slot Message Passing	7
5.1	Architecture Overview	8
5.2	Per-Bead Operations (Node-Local)	8
5.2.1	resetForces()	8
5.2.2	applyGravity(g)	8
5.2.3	onNeighborState(neighborPos, neighborVel, ℓ_0)	8
5.2.4	integrate(dt)	9
5.3	Global Operations (Simulator-Level)	9

5.3.1	Phase 1: Message Dispatch	9
5.3.2	Phase 2: Synchronized Integration	9
5.3.3	Phase 3: Constraint Projection (SHAKE)	9
5.4	Summary: Local vs. Global	10
5.5	Timestep Flowchart	10
6	Validation	12
6.1	Energy Conservation	12
6.2	Constraint Fidelity	12
6.3	Qualitative Behavior	12
7	Next Steps: Binary-Tree GNN	12
7.1	What a Flat GNN Can Already Do	12
7.2	What a Flat GNN Cannot Do	12
7.3	What the Binary Tree Provides	13
7.4	The Research Question	13
7.5	Graph Construction	13
7.6	Message Passing on the Tree	14
7.7	Relation to Constraint Projection	14
7.8	Model Architecture and Data Flow	14
8	Changelog	16

1 Introduction and Motivation

The goal of this project is to:

1. Build a physically accurate FEA-style simulation of a flexible chain/whip system.
2. Use the simulation to generate graph-structured training data.
3. Train a GNN to act as a surrogate solver, learning the physics from data rather than from explicit equations.

The starting problem is a **bead chain released under gravity**: a chain of N mass nodes connected by rod elements, fixed to the ceiling at one end, stretched horizontally, and released. This is a classical benchmark that produces rich nonlinear dynamics (large rotations, wave propagation, pendulum-like swinging) while remaining simple enough to validate.

2 Problem Setup

2.1 Geometry and Initial Conditions

- $N = 16$ mass nodes connected by $N - 1 = 15$ rod elements. (Power of 2 to support binary-tree GNN architecture.)
- Total chain length $L = 2.0$ m, rest length per element $\ell_0 = L/(N - 1)$.
- Total mass $M = 0.5$ kg, distributed with a linear taper. Taper ratio $r = m_0/m_{N-1}$ controls the mass gradient from base to tip ($r = 1$ for uniform, $r = 10$ for whip-like taper).
- Node 0 is fixed at the origin (ceiling anchor).
- Nodes $1 \dots N - 1$ are initially placed horizontally: $\mathbf{x}_i(0) = (i \cdot \ell_0, 0)$.
- Zero initial velocity: $\mathbf{v}_i(0) = \mathbf{0}$.
- Released under gravity $g = 9.81 \text{ m/s}^2$ acting in $-y$.
- Global velocity drag $\mathbf{F}_{\text{drag}} = -\beta \mathbf{v}$ with $\beta = 0.02$ provides gentle settling.

2.2 Graph Structure

The finite element mesh is inherently a graph:

- **Nodes**: mass points with features $(\mathbf{x}, \mathbf{v}, \text{node_type})$.
- **Edges**: rod elements with features $(\ell_0, \text{current stretch})$.
- Edges stored bidirectionally for GNN message passing: $(i \rightarrow j)$ and $(j \rightarrow i)$ for each element.

This maps directly to a `torch_geometric.data.Data` object for GNN training.

3 Formulation Attempts

We explored two fundamentally different approaches to enforcing the near-inextensibility of the rod elements. This section documents both, including what failed and why.

3.1 Attempt 1: Position Verlet with SHAKE Constraints

3.1.1 Formulation

The first approach treated the rods as *rigid constraints* (exact inextensibility) using the SHAKE algorithm paired with position-form Störmer-Verlet integration:

$$\mathbf{x}_{n+1} = 2\mathbf{x}_n - \mathbf{x}_{n-1} + \Delta t^2 \mathbf{a}_n \quad (1)$$

$$\text{SHAKE: } |\mathbf{x}_{n+1,j} - \mathbf{x}_{n+1,i}| = \ell_0 \quad \forall \text{ element } (i, j) \quad (2)$$

SHAKE iteratively projects positions onto the constraint manifold using Gauss-Seidel sweeps along the chain:

$$\boldsymbol{\delta} = \mathbf{x}_j - \mathbf{x}_i, \quad d = |\boldsymbol{\delta}| \quad (3)$$

$$\text{correction} = \frac{d - \ell_0}{d} \boldsymbol{\delta} \quad (4)$$

$$\mathbf{x}_i += \frac{w_i}{w_i + w_j} \text{correction}, \quad \mathbf{x}_j -= \frac{w_j}{w_i + w_j} \text{correction} \quad (5)$$

where $w_i = 1/m_i$ is the inverse mass ($w_0 = 0$ for the fixed anchor).

3.1.2 Results

- **Constraint satisfaction:** Good. Maximum relative length error $\sim 3 \times 10^{-5}$ with 20 Gauss-Seidel iterations per timestep.
- **Energy conservation: Poor.** Total energy drifted by ~ 1 J over 5 s, even with $\Delta t = 0.0005$ s and 100 constraint iterations.

3.1.3 Why It Failed

The energy drift stemmed from the interaction between SHAKE and the velocity update. Two specific issues:

1. **Velocity Verlet variant:** Our initial implementation used velocity Verlet with SHAKE, but the standard SHAKE algorithm only corrects positions—it does not include the RATTLE velocity projection step needed to keep velocities tangent to the constraint manifold. Without RATTLE, the velocity gains a component along the constraint gradient each timestep, injecting energy.
2. **Position Verlet variant:** Switching to pure position Verlet (Eq. 1) avoids explicit velocities, but the Gauss-Seidel SHAKE projection converges slowly for chains: information propagates only one link per iteration. With 19 links, even 100 iterations leave residual constraint errors that accumulate over thousands of timesteps.

3.1.4 Lessons Learned

- SHAKE/RATTLE is well-suited to molecular dynamics where bonds are short and constraint graphs are sparse, but performs poorly on long serial chains.
- For a chain of N nodes, convergence requires $O(N)$ iterations per step, making it inefficient.
- The lack of a tunable stiffness parameter is elegant but makes debugging and validation harder.

3.2 Attempt 2: Symplectic Euler with Penalty Springs

3.2.1 Formulation

The second (and successful) approach replaces rigid constraints with stiff Hookean springs. Each rod element exerts a force:

$$\mathbf{F}_{ij}^{\text{spring}} = k(d - \ell_0) \hat{\boldsymbol{\delta}} \quad (6)$$

where k is the spring stiffness, $d = |\mathbf{x}_j - \mathbf{x}_i|$, and $\hat{\boldsymbol{\delta}} = (\mathbf{x}_j - \mathbf{x}_i)/d$.
Optional viscous damping along the rod axis:

$$\mathbf{F}_{ij}^{\text{damp}} = c((\mathbf{v}_j - \mathbf{v}_i) \cdot \hat{\boldsymbol{\delta}}) \hat{\boldsymbol{\delta}} \quad (7)$$

Time integration uses symplectic (semi-implicit) Euler:

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \Delta t \mathbf{a}(\mathbf{x}_n) \quad (8)$$

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta t \mathbf{v}_{n+1} \quad (9)$$

Note the key difference from explicit Euler: the position update uses the *new* velocity $\mathbf{v}_{n+1}$, not $\mathbf{v}_n$. This makes the scheme symplectic.

3.2.2 Walkthrough: Updating a Single Bead

To build intuition, we trace exactly what happens to one interior bead i (not the fixed anchor) during a single timestep. At the start of the step, bead i has position $\mathbf{x}_i$, velocity $\mathbf{v}_i$, and mass m .

Step 1: Compute all forces on bead i . Three types of force act on it:

1. **Gravity**, pulling it downward:

$$\mathbf{F}_i^{\text{grav}} = (0, -mg) \quad (10)$$

2. **Spring from the left neighbor** (bead $i-1$). The rod connecting them has rest length ℓ_0 . If the current distance $d = |\mathbf{x}_i - \mathbf{x}_{i-1}|$ differs from ℓ_0 , the spring pulls them back together (or pushes them apart):

$$\mathbf{F}_i^{\text{left}} = -k(d - \ell_0) \hat{\boldsymbol{\delta}} \quad (11)$$

where $\hat{\boldsymbol{\delta}}$ points from bead $i-1$ to bead i . When the rod is stretched ($d > \ell_0$), this force pulls bead i back toward its neighbor.

3. **Spring from the right neighbor** (bead $i+1$). Same formula, opposite direction.

An optional viscous **damping** force along each rod resists relative motion between neighbors. The total force is:

$$\mathbf{F}_i = \mathbf{F}_i^{\text{grav}} + \mathbf{F}_i^{\text{left}} + \mathbf{F}_i^{\text{right}} + \mathbf{F}_i^{\text{damp}} \quad (12)$$

Step 2: Acceleration from Newton's second law:

$$\mathbf{a}_i = \frac{\mathbf{F}_i}{m} \quad (13)$$

Step 3: Update velocity *first*.

$$\mathbf{v}_i^{\text{new}} = \mathbf{v}_i + \Delta t \mathbf{a}_i \quad (14)$$

Step 4: Update position using the *new* velocity.

$$\mathbf{x}_i^{\text{new}} = \mathbf{x}_i + \Delta t \mathbf{v}_i^{\text{new}} \quad (15)$$

The ordering—velocity first, then position with the updated velocity—is what makes this *symplectic* Euler rather than plain (explicit) Euler. A small difference in code, but it is what keeps the total energy from drifting over long simulations.

Numerical example. Suppose bead 5 is at $\mathbf{x} = (0.5, -0.3)$ with velocity $\mathbf{v} = (0.1, -2.0)$ m/s, and the total force on it works out to $\mathbf{F} = (3.0, -4.0)$ N with $m = 0.025$ kg:

$$\mathbf{a} = \frac{(3.0, -4.0)}{0.025} = (120, -160) \text{ m/s}^2 \quad (16)$$

$$\mathbf{v}^{\text{new}} = (0.1, -2.0) + 0.0001 \times (120, -160) = (0.112, -2.016) \text{ m/s} \quad (17)$$

$$\mathbf{x}^{\text{new}} = (0.5, -0.3) + 0.0001 \times (0.112, -2.016) = (0.500011, -0.300202) \text{ m} \quad (18)$$

Each bead moves a tiny amount per step. With $\Delta t = 0.0001$ s and a 5 s simulation, we take 50,000 such steps. Each step is the same simple sequence: forces $\rightarrow$ acceleration $\rightarrow$ new velocity $\rightarrow$ new position.

3.2.3 Parameters

Parameter	Value	Notes
Stiffness k	10,000 N/m	High enough for $< 1\%$ stretch
Damping c	0.5	Physical energy dissipation
Timestep Δt	0.0001 s	Stable for $k = 10^4$
Duration	5.0 s	~ 1.75 swing periods
Save interval	50 steps	1001 output frames

Table 1: Default simulation parameters.

3.2.4 Stability Criterion

For symplectic Euler with a spring of stiffness k and mass m , the stability limit is:

$$\Delta t < 2\sqrt{\frac{m}{k}} = 2\sqrt{\frac{0.025}{10,000}} = 0.00316 \text{ s} \quad (19)$$

Our timestep $\Delta t = 0.0001$ s is well within this limit (factor of $31\times$ safety margin).

3.2.5 Results

- **Energy conservation (undamped, $c = 0$):** Excellent. Total energy oscillates within ± 0.04 J over 5 s with no systematic drift. This is the expected behavior for a symplectic integrator.
- **Energy dissipation (damped, $c = 0.5$):** Total energy decreases monotonically from 0 to ~ -1.6 J, confirming the damping acts as a physical energy sink.
- **Constraint satisfaction:** Maximum rod stretch $< 1\%$ at $k = 10^4$. Occasional peaks to $\sim 3\%$ during high-velocity events (tip snap).
- **Qualitative behavior:** Chain falls, swings through the vertical, overshoots to the opposite side, and gradually settles due to damping. Physically correct.

3.2.6 Why It Works

1. **Symplectic structure:** The semi-implicit Euler scheme preserves the symplectic form of Hamiltonian mechanics, meaning energy oscillates around the true value but doesn't drift.
2. **No iterative projection:** Unlike SHAKE, there is no iterative constraint solve. Forces are computed explicitly from positions, making each timestep $O(N)$ with a small constant.
3. **Tunable stiffness:** The spring constant k provides a direct physical knob. Increasing k reduces stretch at the cost of requiring smaller Δt .
4. **GNN compatibility:** The force computation (Eq. 6) is a local, pairwise message along edges — exactly the computation a GNN learns to approximate.

4 Implementation

4.1 Code Structure

File	Purpose
<code>simulation.py</code>	Chain state, force computation, time integration
<code>visualize.py</code>	Matplotlib animation and energy diagnostics
<code>main.py</code>	CLI runner, parameter config, <code>.npz</code> export

Table 2: Source files.

4.2 Data Export Format

Simulation output is saved as a NumPy `.npz` archive with the following arrays, designed for direct consumption by a GNN:

Key	Shape	Description
<code>positions</code>	$(T, N, 2)$	Node positions per frame
<code>velocities</code>	$(T, N, 2)$	Node velocities per frame
<code>accelerations</code>	$(T, N, 2)$	Node accelerations (training target)
<code>edge_index</code>	$(2, 2E)$	Bidirectional edge connectivity
<code>node_types</code>	$(N,)$	0 = free, 1 = fixed
<code>rest_lengths</code>	$(E,)$	Element rest lengths
<code>times</code>	$(T,)$	Timestamps

Table 3: Data export format. T = saved frames, N = nodes, E = elements.

Scalar metadata (`dt`, `gravity`, `stiffness`, `damping`, `n_nodes`, etc.) is also included.

5 Qt/C++ Implementation: Signal/Slot Message Passing

The Qt implementation mirrors the Python physics exactly but structures the computation as *local per-node operations* coordinated by a central simulator—directly analogous to a GNN's message-passing architecture. This section documents which operations occur inside each bead (node-local) versus which require global coordination.

5.1 Architecture Overview

The simulation consists of three object types:

- **Bead** — a `QObject` representing a single mass node. Each bead owns its position $\mathbf{x}$, velocity $\mathbf{v}$, mass m , and a force accumulator $\mathbf{F}_{\text{accum}}$. Beads communicate via Qt signals and slots.
- **Rod** — a lightweight struct storing edge connectivity (two bead indices) and rest length ℓ_0 . Rods have no behavior; they define the graph topology.
- **ChainSimulator** — the coordinator that owns all beads and rods, drives the timestep, and manages global operations.

5.2 Per-Bead Operations (Node-Local)

Each bead performs the following operations using *only its own state* and *messages received from immediate neighbors*. These are implemented as Qt slots on the **Bead** class:

5.2.1 `resetForces()`

Zeros the force accumulator: $\mathbf{F}_{\text{accum}} \leftarrow \mathbf{0}$.

5.2.2 `applyGravity(g)`

Adds gravitational force using only the bead’s own mass:

$$\mathbf{F}_{\text{accum}} += (0, -mg) \quad (20)$$

If a global drag coefficient $\beta > 0$ is set, also applies velocity drag:

$$\mathbf{F}_{\text{accum}} -= \beta \mathbf{v} \quad (21)$$

Fixed beads skip this entirely.

5.2.3 `onNeighborState(neighborPos, neighborVel, $\ell_0$ )`

This is the **message-passing function**—the GNN analogue of an edge message followed by aggregation. When a bead receives a neighbor’s state, it computes the spring and damping forces from that single neighbor and adds them to its accumulator:

$$\boldsymbol{\delta} = \mathbf{x}_{\text{neighbor}} - \mathbf{x}_{\text{self}} \quad (22)$$

$$d = |\boldsymbol{\delta}|, \quad \hat{\mathbf{d}} = \boldsymbol{\delta}/d \quad (23)$$

$$\mathbf{F}_{\text{accum}} += k(d - \ell_0)\hat{\mathbf{d}} \quad (\text{spring}) \quad (24)$$

$$\mathbf{F}_{\text{accum}} += c((\mathbf{v}_{\text{neighbor}} - \mathbf{v}_{\text{self}}) \cdot \hat{\mathbf{d}})\hat{\mathbf{d}} \quad (\text{damping}) \quad (25)$$

This slot is called once per neighbor per timestep. For an interior bead with two neighbors, the accumulator sums two spring/damping contributions. The bead does *not* know how many neighbors it has or where they are in the chain—it simply accumulates whatever messages arrive.

5.2.4 integrate(dt)

The **node update function**. Using only its own accumulated force and mass, the bead updates its state via symplectic Euler:

$$\mathbf{a} = \mathbf{F}_{\text{accum}}/m \quad (26)$$

$$\mathbf{v} += \Delta t \mathbf{a} \quad (27)$$

$$\mathbf{x} += \Delta t \mathbf{v} \quad (28)$$

Fixed beads zero their force accumulator and return without moving.

5.3 Global Operations (Simulator-Level)

The **ChainSimulator** performs three operations that require knowledge of the full graph structure. These cannot be decomposed into independent per-node computations.

5.3.1 Phase 1: Message Dispatch

The simulator iterates over all rods (edges) and calls **onNeighborState** on both endpoints:

Listing 1: Message dispatch in ChainSimulator::step()

```
1 for (const Rod &rod : m_rods) {  
2     Bead *a = m_beads[rod.beadA];  
3     Bead *b = m_beads[rod.beadB];  
4     b->onNeighborState(a->id(), a->pos(), a->vel(), rod.restLength);  
5     a->onNeighborState(b->id(), b->pos(), b->vel(), rod.restLength);  
6 }
```

This is the coordinator role: the simulator knows the graph topology (which beads are connected) and orchestrates message delivery. The beads themselves are topology-unaware.

5.3.2 Phase 2: Synchronized Integration

After all messages have been delivered and forces accumulated, the simulator triggers integration on all beads simultaneously:

```
1 for (Bead *b : m_beads)  
2     b->integrate(dt);
```

The synchronization matters: all beads must finish accumulating forces before any bead updates its position. This prevents order-dependent results and mirrors the synchronous update in a GNN forward pass.

5.3.3 Phase 3: Constraint Projection (SHAKE)

After integration, the simulator optionally runs an iterative SHAKE projection over all rods to enforce inextensibility. This is fundamentally **non-local**: correcting one rod's length changes the positions of its endpoints, which affects adjacent rods. The algorithm iterates until convergence:

Listing 2: SHAKE constraint projection (simplified)

```
1 for (int iter = 0; iter < maxIters; ++iter) {  
2     for (const Rod &rod : m_rods) {  
3         // read both endpoints, compute length error,  
4         // adjust both positions proportional to inverse mass  
5     }  
6     if (maxError < tolerance) break;
```

```

7 }
8 // then correct velocities along each rod

```

This phase has no direct analogue in a single-layer GNN. It requires:

- Reading and writing *pairs* of nodes simultaneously
- Multiple iterations over the entire edge set
- Convergence-dependent termination

A flat GNN would need $O(N)$ message-passing layers to approximate what SHAKE achieves in ~ 10 iterations. This motivates the hierarchical binary-tree architecture described in Section 7.

5.4 Summary: Local vs. Global

Operation	Scope	GNN Analogue
<code>resetForces()</code>	Node-local	Reset hidden state
<code>applyGravity(g)</code>	Node-local	Node feature (external force)
<code>onNeighborState()</code>	Node-local (1-hop)	Edge message + aggregation
<code>integrate(dt)</code>	Node-local	Node update function
Message dispatch	Global (topology)	GNN layer coordination
Synchronized integration	Global (ordering)	Synchronous update
SHAKE projection	Global (iterative)	No single-layer analogue

Table 4: Classification of simulation operations by scope.

The first four operations—everything inside the `Bead` class—map directly to a standard GNN layer. The last three are coordinator responsibilities. Of these, message dispatch and synchronization are handled by any GNN framework automatically. Only the SHAKE constraint projection requires architectural innovation (hierarchical message passing) to capture.

5.5 Timestep Flowchart

Figure 1 shows a single timestep as a threading diagram. The left lane represents the `Bead` objects (child threads, running in parallel), and the right lane represents the `ChainSimulator` (parent/coordinator thread). Horizontal dashed lines are synchronization barriers where the coordinator waits for all beads to finish before proceeding. Diagonal arrows show data flow between the two lanes.

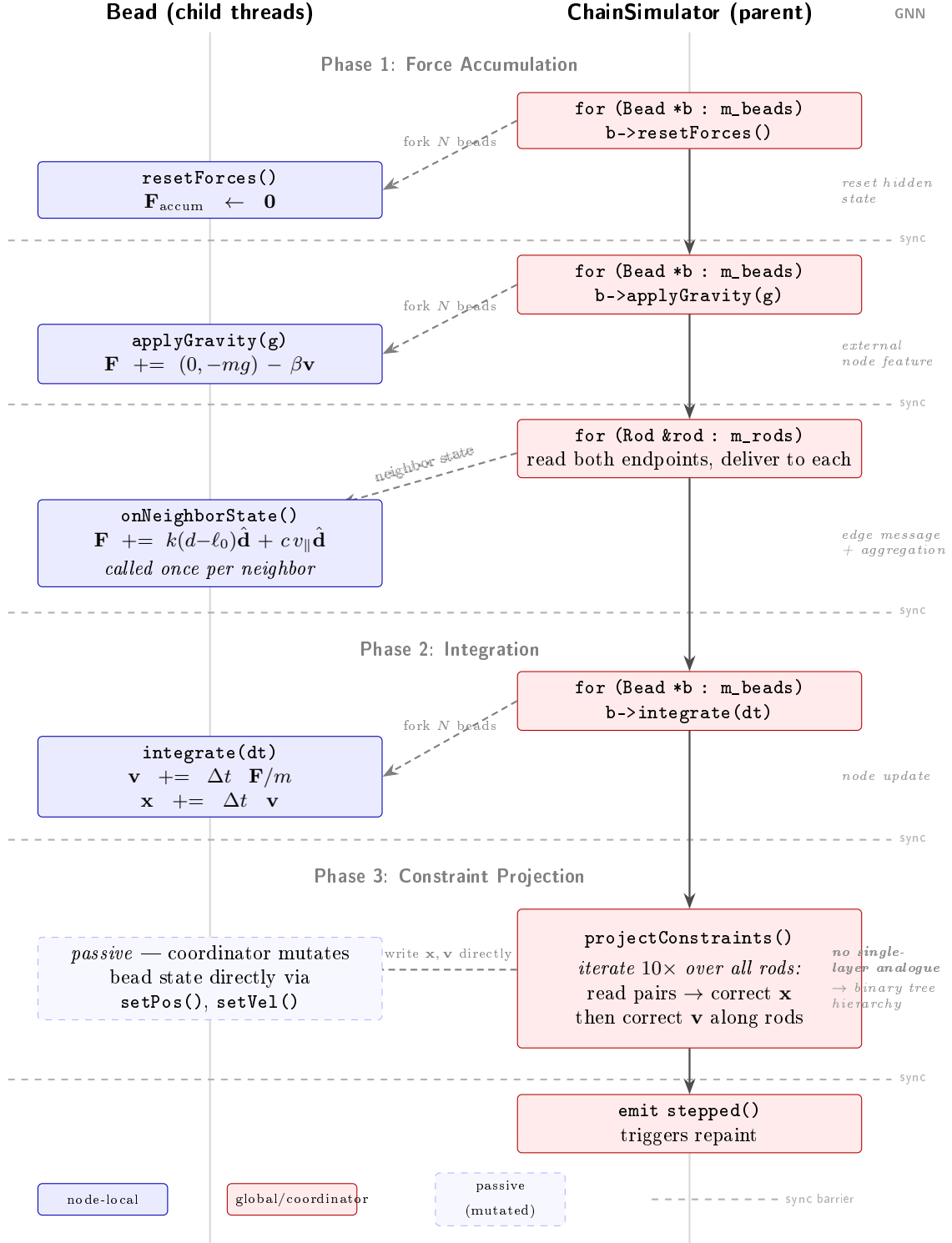


Figure 1: Threading diagram of a single simulation timestep. Left lane: **Bead** objects (child threads) executing node-local operations in parallel. Right lane: **ChainSimulator** (parent thread) coordinating graph-level operations. Dashed horizontal lines are synchronization barriers. In Phase 3, the coordinator directly mutates bead state—beads are passive. Right annotations show the GNN analogue; Phase 3 has no single-layer equivalent.

6 Validation

6.1 Energy Conservation

With zero damping ($c = 0$), the symplectic Euler integrator conserves total energy (kinetic + gravitational potential + elastic potential) to within ± 0.04 J over 5 s of simulation. The energy oscillates symmetrically without drift, confirming the symplectic property.

With damping ($c = 0.5$), total energy decreases monotonically as expected.

6.2 Constraint Fidelity

Rod element lengths remain within 1% of their rest length for most of the simulation, with brief excursions to $\sim 3\%$ during high-velocity tip motion. This is acceptable for GNN training data and can be tightened by increasing k (with proportionally smaller Δt).

6.3 Qualitative Behavior

The chain exhibits the expected physics:

- Initial free-fall of the tip end.
- Wave propagation from the free end toward the anchor.
- Pendulum-like swinging with the chain curving under inertia.
- Gradual damping to the vertical equilibrium.

7 Next Steps: Binary-Tree GNN

This section frames the central research question and describes the proposed architecture.

7.1 What a Flat GNN Can Already Do

Phases 1 and 2 of the simulation (Section 5)—force accumulation and integration—are a natural fit for a standard GNN. A single message-passing layer can learn `onNeighborState()` (spring and damping forces from immediate neighbors) and `integrate()` (converting accumulated force to velocity and position via F/m). These are purely local, per-node operations that depend only on 1-hop neighbor state. A flat GNN trained on simulation data could learn these operations today.

7.2 What a Flat GNN Cannot Do

The physics of the bead chain has three properties that are fundamentally **non-local**—no single bead can compute or verify them from its own state and immediate neighbors alone:

1. **Inextensibility.** Correcting one rod’s length moves its endpoints, which violates the length constraints of adjacent rods, which need correcting too. A single correction propagates through the entire chain. The SHAKE solver handles this with ~ 10 iterations over all rods, but a 1-layer GNN has no mechanism for iterative global correction.
2. **Energy conservation.** Total energy $E = \sum_i \frac{1}{2} m_i |\mathbf{v}_i|^2 + m_i g y_i + \frac{1}{2} k \sum_e (\ell_e - \ell_0)^2$ is a sum over *all* nodes and edges. No single bead can know whether the system is conserving energy. If bead 15 gains kinetic energy, bead 1’s update should be consistent with that—but in a flat GNN, bead 1 cannot see bead 15.

3. **Momentum conservation.** Total momentum $\mathbf{p} = \sum_i m_i \mathbf{v}_i$ is similarly global. Local updates that individually look reasonable can collectively violate momentum conservation.

A flat GNN with K message-passing layers has a receptive field of K hops. For a 16-bead chain, the tip needs 15 hops to influence the anchor—requiring 15 layers, which is impractical (vanishing gradients, overfitting, cost).

7.3 What the Binary Tree Provides

The tree does *not* replace any of the existing local operations. Beads still compute spring forces, damping, and integration exactly as before. The tree adds a **communication backbone** that lets global information flow in $O(\log N)$ hops instead of $O(N)$:

- The **root node** can learn a representation of the full system state—this is where conservation laws (energy, momentum) can be enforced.
- The **interior nodes** can learn corrections that propagate back down to the beads: “your local update would violate a global constraint; adjust by this much.”
- The **bead nodes** still perform their local force computation exactly as before—the tree augments, it does not replace.

The tree is not an approximation of SHAKE. It gives the GNN the **architectural capacity** to learn whatever global operation is needed—whether that is constraint projection, energy conservation, or something else the network discovers during training.

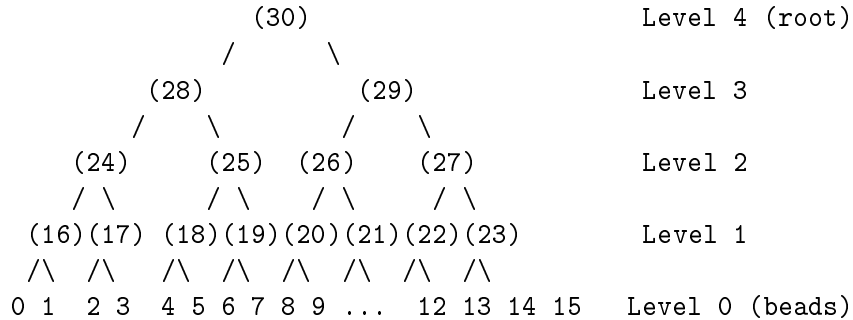
7.4 The Research Question

*Can a GNN with a binary-tree communication backbone learn to produce physically accurate trajectories—correct local forces **and** global conservation—in a single forward pass, without an explicit iterative constraint solver?*

If yes, this demonstrates that hierarchical message passing is sufficient to replace iterative global solvers, with implications for GNN surrogates in any domain where local physics is coupled with global constraints (fluid simulation, structural mechanics, molecular dynamics).

7.5 Graph Construction

The chain uses $N = 16$ beads (power of 2). A binary tree is built on top:



- **16 bead nodes** (Level 0): physical state $(\mathbf{x}, \mathbf{v}, m)$
- **15 interior nodes** (Levels 1–4): learned feature vectors, no physical state
- **31 total nodes**, connected by bead-chain edges (Level 0) plus parent-child tree edges (all levels)

7.6 Message Passing on the Tree

All edges—bead-to-bead, bead-to-interior, interior-to-interior—use the same message-passing mechanism. There are no pooling or unpooling operations; interior nodes are simply additional nodes in the graph that participate in standard message passing.

- **Up-pass:** children send messages to parents, building regional and then global summaries.
- **Down-pass:** parents send corrections to children, communicating global constraints back to individual beads.
- **Chain edges:** beads exchange local spring/damping messages as before.

Any bead-to-bead path through the tree is at most $2\log_2 N = 8$ hops (up 4, down 4), compared to 15 hops along the chain. The root node sees the entire chain, enabling enforcement of global conservation laws (energy, momentum) in a single forward pass.

7.7 Relation to Constraint Projection

The SHAKE solver is essentially a Gauss-Seidel relaxation—slow because each rod correction disturbs its neighbors. The binary tree provides a multigrid-like structure:

- Level 0 edges handle local rod corrections
- Level 1 nodes coordinate pairs of adjacent rods
- Higher levels coordinate progressively larger chain segments
- The root coordinates the entire chain

A trained GNN on this hierarchical graph should learn to approximate the iterative SHAKE projection in a single forward pass through the tree.

7.8 Model Architecture and Data Flow

Figure 2 shows the full U-Net model with tensor shapes at each stage. The model takes the unconstrained state (after integration, before SHAKE) and predicts the position and velocity corrections that SHAKE would compute.

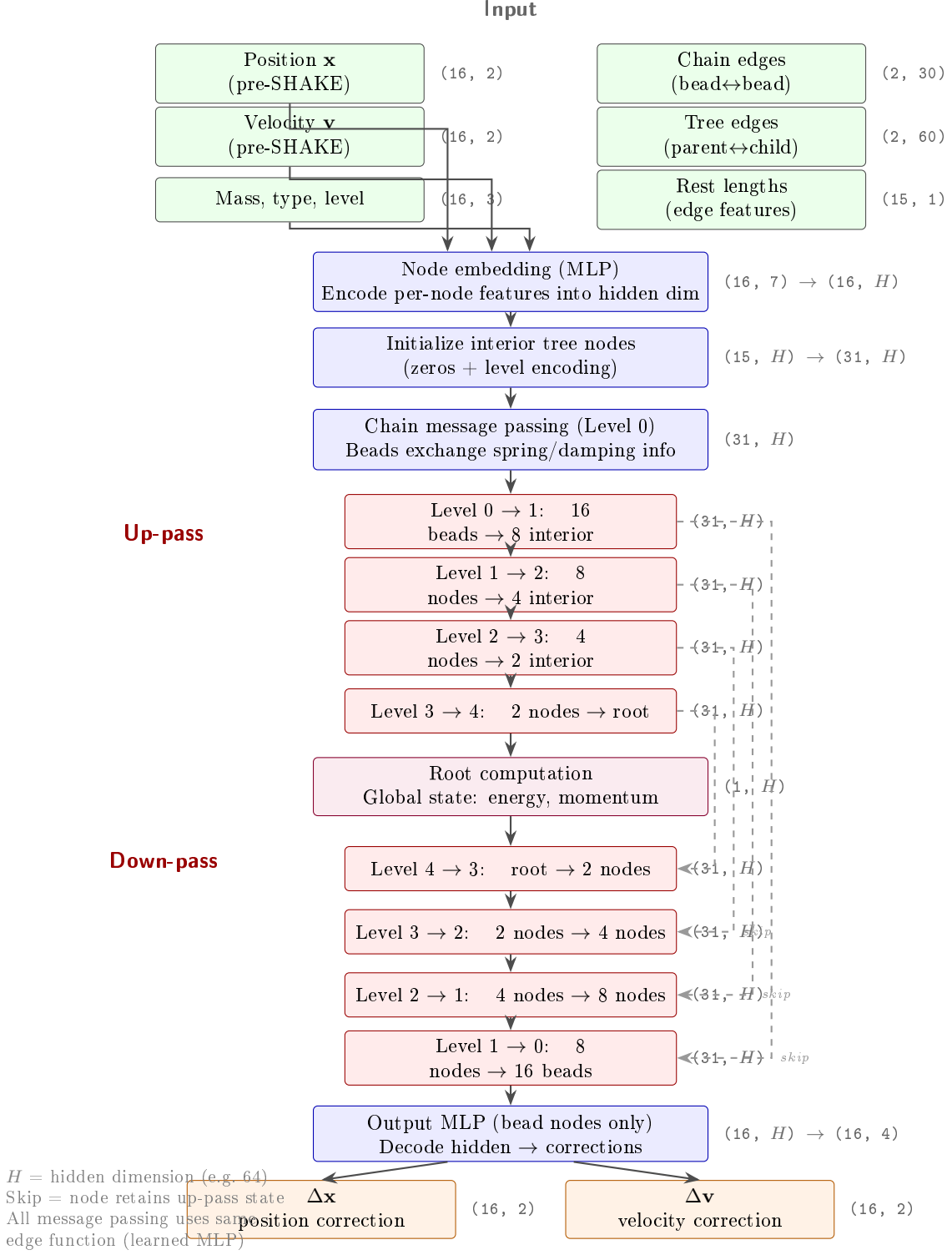


Figure 2: U-Net GNN architecture with tensor shapes. Green boxes: input data. Blue boxes: processing steps. Red boxes: tree message-passing levels. Purple box: root computation (global). Orange boxes: output predictions. Dashed arrows indicate skip connections (each node retains its up-pass state for use during the down-pass). Hidden dimension H is a hyperparameter. The model takes the unconstrained state after integration and predicts the SHAKE corrections ($\Delta\mathbf{x}, \Delta\mathbf{v}$) for each bead.

8 Changelog

2026-03-14 Added Qt/C++ architecture section documenting per-bead (node-local) vs. global operations with GNN analogues. Added binary-tree GNN plan for hierarchical message passing. Updated simulation to 16 beads, tapered mass, SHAKE constraints, and global drag.

2026-03-12 Initial report. Documented SHAKE (failed) and symplectic Euler (successful) formulations. Simulation validated for energy conservation and constraint fidelity. GNN-ready data export implemented.